

# **Taller de Proyecto II**

## **Informe Final**

### **J6: Plataforma Multi-Touch con Celdas de Carga**

#### **Grupo de Desarrollo**

|                          |           |
|--------------------------|-----------|
| Dappiano, Máximo         | [03210/8] |
| Restucha, Tiago          | [03398/7] |
| Schwindt, Ignacio Andres | [03229/0] |

Fecha de entrega: 09/12/25

# 1. Descripción General del Proyecto

El presente proyecto consiste en el diseño, desarrollo e implementación de una plataforma táctil de bajo costo basada en tecnología de celdas de carga. El sistema permite la detección de variables físicas fundamentales como la magnitud de la fuerza aplicada, la detección de pulsos y el cálculo de coordenadas bidimensionales ( $X, Y$ ) de objetos sobre una superficie rígida.

## 1.1. Propuesta Original y Motivación

Originalmente, la propuesta se inspiró en el sistema *SCALE* desarrollado por el MIT Media Lab. Dicho sistema utiliza redes densas de sensores distribuidos para capturar tanto la magnitud como la topología de la fuerza aplicada. El objetivo inicial de nuestro grupo planteaba replicar este concepto para crear una superficie capaz de reconocer interacciones complejas y gestuales en dos dimensiones, funcionando de manera análoga a un panel táctil o “mousepad” de gran escala, pero diferenciándose por su capacidad de medir la presión.

## 1.2. Evolución y Modificaciones por Evaluaciones de Cátedra

A lo largo del ciclo de desarrollo, y tras la presentación del “plan de proyecto” inicial, se realizaron ajustes estratégicos validados por el equipo docente para asegurar la viabilidad técnica y temporal:

- **Adelantamiento de la Etapa de Hardware:** Se modificó el cronograma para priorizar el ensamblaje físico sobre la simulación. Se detectó que las herramientas de simulación (como Proteus) no podían emular fielmente la respuesta mecánica y el ruido eléctrico inherente a las celdas de carga trabajando en conjunto (configuración triangular), haciendo indispensable la validación empírica temprana.
- **Redefinición del Cronograma y Alcance:** Se debió reprogramar la etapa de pruebas debido a incidentes técnicos con la fuente de alimentación provista por la cátedra (sobretensión de 16V). Esto ocasionó la avería irreversible de dos módulos HX711 y daños en las celdas de carga. Como medida de contingencia, se compraron nuevamente los sensores y se retrasó la etapa de pruebas conjuntas para diciembre.

## 1.3. Decisiones de Diseño y Arquitectura Final

Para la consolidación del producto final entregado, se tomaron decisiones técnicas clave:

- **Comunicación vía WebSockets:** Se descartaron protocolos tradicionales como HTTP polling debido a su alta latencia. Se implementó un servidor WebSocket en el ESP8266 para garantizar una transmisión de datos en tiempo real (*full-duplex*), esencial para una respuesta visual fluida ante el movimiento del usuario.
- **Interfaz Reactiva (React.js):** Se seleccionó la biblioteca React para el frontend debido a su eficiencia en el manejo del DOM virtual. Esto permite renderizar vectores de movimiento y rastros de gestos a altas tasas de refresco (60 FPS) sin penalizar el rendimiento del navegador.
- **Geometría Triangular:** Se optó por una disposición de sensores en triángulo equilátero. Esta geometría es la mínima necesaria para definir un plano y permite calcular coordenadas baricéntricas de manera eficiente, optimizando el uso de pines GPIO en el microcontrolador y reduciendo el costo total del hardware.

## 2. Presentación Esquemática del Proyecto

### 2.1. Diagrama de Flujo de Procesos y Funciones

El sistema opera mediante una cadena de procesamiento de señales que va desde el dominio físico al digital. Las celdas de carga, al deformarse mecánicamente, alteran su resistencia eléctrica (puente de Wheatstone). Estas variaciones infinitesimales son capturadas por los módulos HX711, que envían la señal digitalizada al microcontrolador.

El firmware del ESP8266 calcula el Centro de Presión (CoP) y empaqueta la información en objetos JSON. Finalmente, la interfaz web decodifica estos paquetes para la visualización.

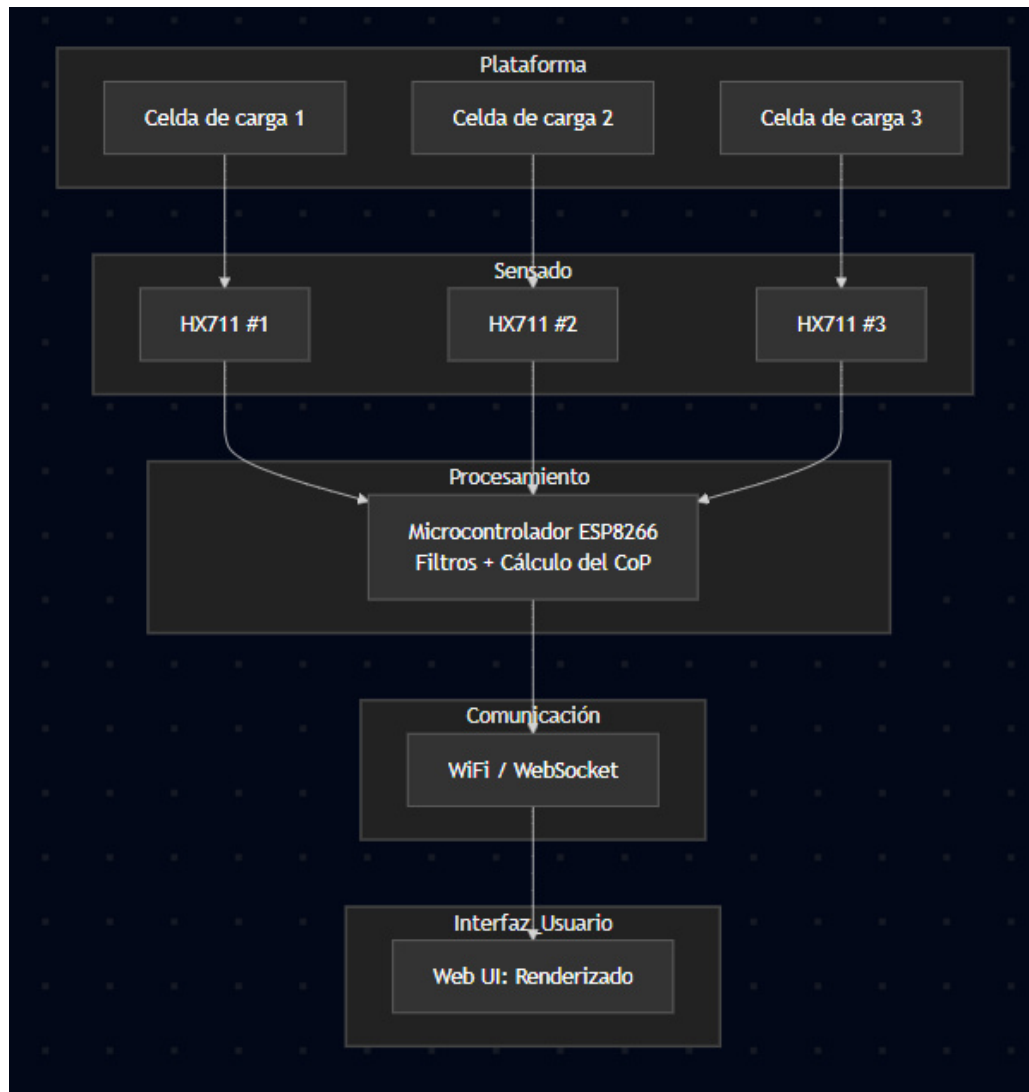


Figura 1: Diagrama de Bloques y Procesos

El sistema utiliza tres celdas de carga en el Hardware cuya señal analógica es digitalizada por la librería HX711 Lib. El Firmware emplea `scales.ino` para obtener estas lecturas y `comision.ino` para la conectividad, usando librerías como `ESPAsyncWebServer` para establecer un WebSocket que envía las coordenadas  $(x, y)$  y la magnitud  $(w)$ . Finalmente, la Webinterface (Front-end) recibe estos datos a través de un WebSocket Hook en `App.jsx`, los procesa con funciones internas (Math) y los usa para el Renderizado SVG de la visualización.

A continuación se muestra el diagrama de las funciones

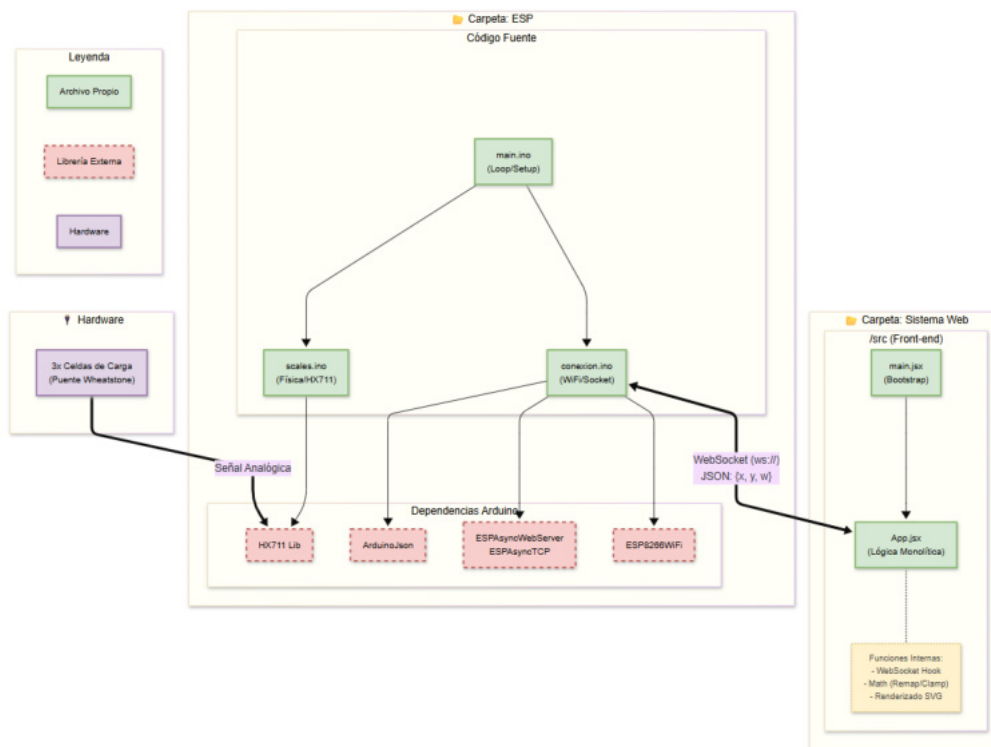


Figura 2: Diagrama de interacción de librerías

## 2.2. Esquemático de Conexiones de Hardware

El diseño electrónico se centra en la interconexión del microcontrolador ESP8266 (NodeMCU) con tres módulos amplificadores HX711. Se incluye una etapa de alimentación regulada para garantizar la estabilidad de las lecturas, dado que las celdas de carga son sensibles a fluctuaciones de voltaje.

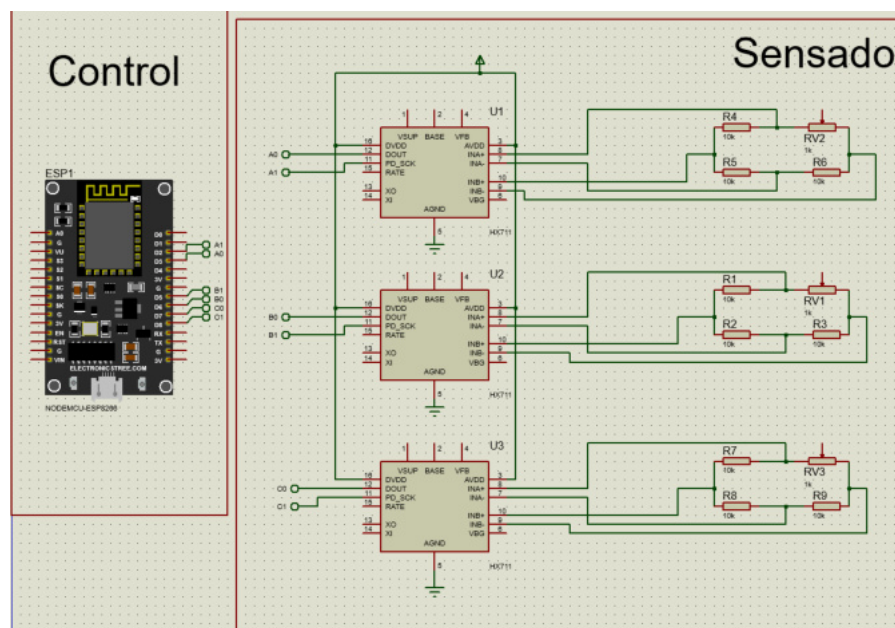


Figura 3: Esquemático Proteus

## 2.3. Evaluación de Funcionalidades y Requerimientos

Al cierre del proyecto, todos los objetivos planteados han sido alcanzados satisfactoriamente. El estado de cumplimiento es el siguiente:

- **Estructura Física (100 %):** Se desarrolló una estructura rígida que aloja las tres celdas de carga distribuidas uniformemente en los extremos. El diseño garantiza la estabilidad mecánica requerida para la medición precisa.
- **Electrónica de Lectura (100 %):** La implementación de los módulos HX711 junto con el microcontrolador permite la lectura simultánea y eficiente de las celdas, cumpliendo con los requisitos de adquisición de datos.
- **Algoritmos de Posición (100 %):** Los algoritmos diseñados calculan correctamente la posición bidimensional de la carga combinando las lecturas de las tres celdas. El sistema se encuentra calibrado y es capaz de determinar las coordenadas (X, Y) con precisión sobre la superficie.
- **Detección de Gestos (100 %):** Se logró distinguir exitosamente entre gestos de “arrastre” (*drag*) y “presión” (*push*). La lógica implementada detecta estos eventos y los clasifica correctamente para su visualización.
- **Interfaz Web (100 %):** La plataforma web muestra en tiempo real la posición del objeto y los gestos detectados, cumpliendo con el objetivo de visualización y monitoreo remoto.
- **Validación y Documentación (100 %):** Se validó el rendimiento del prototipo mediante pruebas sistemáticas y se generó la documentación técnica necesaria para garantizar la reproducibilidad y el escalado del diseño.

## 3. Documentación Técnica del Software

El desarrollo de software se ha modularizado en dos grandes componentes: el firmware embebido y la interfaz de usuario. A continuación, se detalla la arquitectura del repositorio entregado.

### 3.1. Estructura de Archivos y Directorios

**1. Directorio /Firmware:** Código fuente C++ para el ESP8266.

- `main.ino`: Punto de entrada del sistema. Orquesta la inicialización y el bucle principal.
- `scales.ino`: Módulo de gestión de hardware y física del proyecto.
- `conexion.ino`: Módulo de conectividad y comunicación de red.

Tabla 1: Funciones y Procesos Implementados por el Grupo de Desarrollo

| Archivo      | Función / Proceso | Descripción                                                                                                                             |
|--------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| scales.ino   | initScales()      | Inicializa los pines de comunicación y realiza el tarado y calibración inicial de las galgas HX711.                                     |
| scales.ino   | updateScales()    | Adquiere los datos crudos, calcula el peso total ( $W$ ) y determina las coordenadas ( $X, Y$ ) mediante triangulación.                 |
| conexion.ino | initWiFi()        | Establece la conexión a la red inalámbrica local en modo estación (STA) y verifica la asignación de IP.                                 |
| conexion.ino | initWebSocket()   | Configura el servidor asíncrono, asigna los manejadores de eventos y levanta el servicio en el puerto 80.                               |
| conexion.ino | onWsEvent(...)    | Gestiona los eventos del protocolo WebSocket: conexión de clientes, desconexión y recepción de mensajes de control.                     |
| conexion.ino | sendPositionWS()  | Serializa las variables globales calculadas ( $g_x, g_y, g_W$ ) en formato JSON y las transmite a todos los clientes conectados (10Hz). |
| main.ino     | setup()           | Función de orquestación que llama secuencialmente a las inicializaciones de comunicación y hardware.                                    |
| main.ino     | loop()            | Ciclo principal que ejecuta la lectura de sensores y el envío de datos de forma repetitiva.                                             |

**Directorio /WebInterface:** Aplicación cliente desarrollada con React.js y Vite.

- /src/App.jsx: Componente principal que centraliza toda la lógica del frontend. Gestiona la conexión WebSocket para la recepción de datos en tiempo real y utiliza gráficos vectoriales (SVG) para renderizar visualmente la posición del cursor y su historial de movimiento (rastros), sin depender de librerías gráficas externas complejas.

### 3.2. Guía de Replicación e Instalación (Entorno Arduino)

Para reproducir el entorno de desarrollo y cargar el firmware:

#### 1. Configuración del IDE:

- Instalar el gestor de tarjetas *esp8266 by ESP8266 Community* (v3.0.2+).
- Seleccionar placa: NodeMCU 0.9 (ESP-12 Module).
- Configuración de carga: Velocidad 115200, Frecuencia CPU 80 MHz.

#### 2. Gestión de Dependencias: Es necesario instalar las siguientes bibliotecas externas:

- **HX711 (Bogdan Necula):** Driver estándar para la comunicación con el conversor ADC de 24 bits.
- **ESPAsyncWebServer** y **ESPAsyncTCP:** Librerías fundamentales para levantar el servidor web y el socket de forma asíncrona, evitando que la comunicación bloquee el muestreo de los sensores.
- **ArduinoJson (Benoit Blanchon):** Utilizada para la creación y serialización eficiente de los paquetes de datos en formato JSON.

3. **Configuración de Red:** El archivo `conexion.ino` controla el modo de operación. Actualmente, el sistema está configurado mediante `WiFi.mode(WIFI_STA)` para conectarse a una red existente.

### 3.3. Análisis de Caso de Uso: Interacción en Tiempo Real

**Escenario:** Un usuario desliza el dedo desde el centro hacia el borde superior de la plataforma.

**Secuencia Lógica de Operación:**

1. **Física:** La presión del dedo altera el equilibrio de momentos de fuerza sobre la placa triangular. El sensor superior registra un aumento de carga proporcional, mientras los inferiores registran valores menores.
2. **Adquisición:** Los tres módulos HX711 muestrean la señal a 5Hz.
3. **Procesamiento (ESP8266):**
  - Se leen los valores crudos.
  - Se restan los valores de tara (offset).
  - Se calcula el peso total  $W_T = w_1 + w_2 + w_3$ .
  - Se calculan las coordenadas  $(X, Y)$  usando las posiciones relativas de los sensores.
  - Se serializa el paquete JSON: `{'x': 0.0, 'y': 15.5, 'w': 250}`.
4. **Transmisión:** El paquete se envía vía WebSocket Broadcast a todos los clientes conectados.
5. **Renderizado (Web):** El navegador recibe el evento, parsea el JSON y actualiza las coordenadas del componente `Canvas`, dibujando el nuevo punto y añadiendo el anterior al historial de rastro.

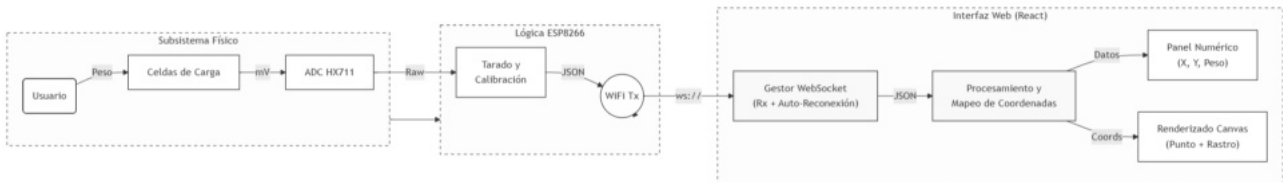
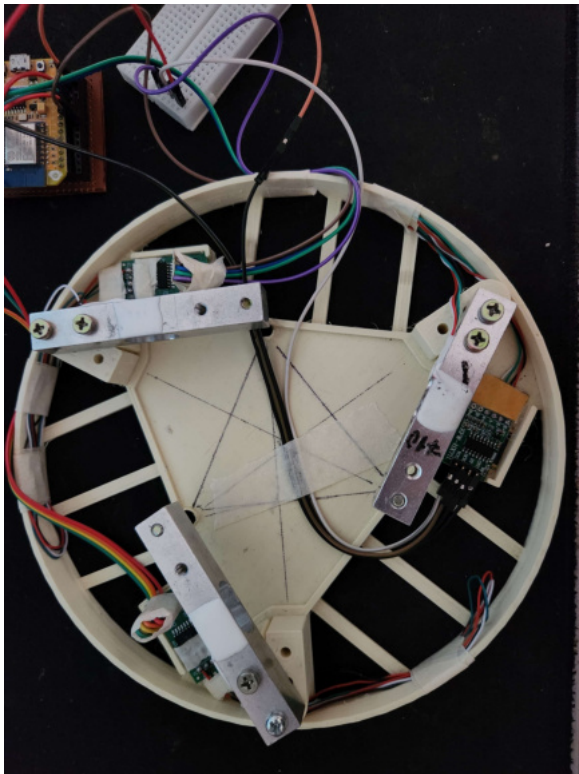


Figura 4: Flujo completo de datos desde la interacción física hasta la retroalimentación visual

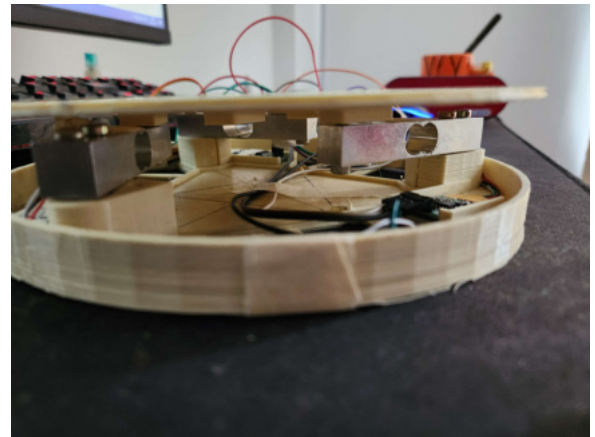
## 4. Otra Documentación Relacionada

### 4.1. Registro Visual del Prototipo

Se adjuntan evidencias del ensamblaje final y de la interfaz en funcionamiento.

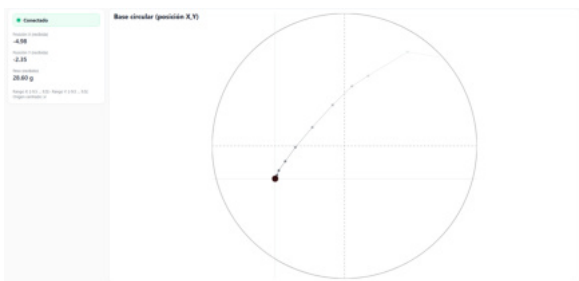


(a) Ubicacion Galgas

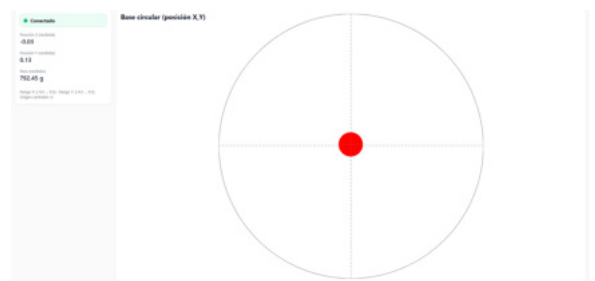


(b) Armado Completo

Figura 5: Estructura Fisica



(a) Detección Drag



(b) Detección Push

Figura 6: Pagina Web Funcionando

## 4.2. Recursos Digitales y Repositorios

Para acceder a la totalidad de los recursos, código fuente y documentación histórica, referirse a los siguientes enlaces:

- **Video Demostrativo:** <https://photos.google.com/share/AF1QipMStrf7CQVsIgAoXlGP06SizXRjenOhMfRWykVzluxVFSLBW7FUiP3g?key=c1lGOVlOMXlnR0xTRXgxVWtKTlhQYzFaajhfUDZB> (Duración: 2 min).
- **Bitácora de Desarrollo:** <https://github.com/tpII/2025-J6-Superficie-Interactiva/blob/master/Bitacora/bitacora.txt> (Registro cronológico de avances y obstáculos).
- **Repositorio de Código:** <https://github.com/tpII/2025-J6-Superficie-Interactiva/tree/master> (Código fuente completo de Firmware y Web).



## A. Anexo: Materiales y Presupuesto Ejecutado

A continuación se presenta el desglose final de costos. Se utiliza un código de colores para identificar el origen de los fondos: Verde para material provisto por la cátedra, Rojo para inversiones del grupo, y Naranja para ítems mixtos o reposiciones parciales por parte de los estudiantes (específicamente 2 celdas de carga reemplazadas). Precios actualizados a Diciembre 2025.

Tabla 2: Costos asociados al proyecto

| Material                       | Cantidad | Precio Unitario | Precio Total |
|--------------------------------|----------|-----------------|--------------|
| ESP8266 (NodeMCU)              | 1        | \$6.600         | \$6.600      |
| Kit Hx711 + Galga 1kg          | 3        | \$9.800         | \$29.400     |
| Cableado multipropósito        | 12       | \$170           | \$2.040      |
| Servicio de Impresión 3D (PLA) | 2        | \$9.000         | \$18.000     |
| Tornillería y fijaciones       | 12       | \$200           | \$2.400      |
| Fuente Switching AC/DC 5V 1A   | 1        | \$4.039         | \$4.039      |
| Total                          |          |                 | \$62.479     |